

EXPERIMENT – 06

Date: 18/09/2026

Java Collection Framework: Important Methods by Class

1. AIM / OBJECTIVE

To study and implement the important methods of commonly used Java Collection Framework classes such as ArrayList, LinkedList, Vector, Stack, HashSet, LinkedHashSet, TreeSet, PriorityQueue, ArrayDeque, HashMap, LinkedHashMap, TreeMap, and Hashtable.

2. PROBLEM STATEMENT

Write a Java program to create and use different Collection Framework classes and demonstrate their important methods for adding, accessing, updating, removing, searching, sizing, sorting, navigating, and working with key-value pairs.

3. CONCEPTS USED

List, Set, Queue, Deque, Map, Generics, array-based collections, linked collections, hash-based collections, sorted collections, key-value pairs, Stack (LIFO), and Priority Queue.

4. COLLECTION CLASSES USED

List Classes	ArrayList, LinkedList, Vector, Stack
Set Classes	HashSet, LinkedHashSet, TreeSet
Queue / Deque Classes	PriorityQueue, ArrayDeque
Map Classes	HashMap, LinkedHashMap, TreeMap, Hashtable

Note: Map is separate from the Collection hierarchy because it stores key-value pairs.

5. IMPORTANT CLASS BEHAVIOUR

Class	Main Feature
ArrayList	Dynamic array, maintains insertion order
LinkedList	Doubly linked list, supports list and queue operations
Vector	Dynamic array, synchronized legacy class
Stack	LIFO data structure
HashSet	Unique elements, no guaranteed order
LinkedHashSet	Unique elements + insertion order
TreeSet	Unique elements + sorted order
PriorityQueue	Priority-based queue
ArrayDeque	Double-ended queue
HashMap	Key-value pairs, no guaranteed key order
LinkedHashMap	Key-value pairs + insertion order
TreeMap	Key-value pairs + sorted keys
Hashtable	Synchronized legacy key-value collection

6. IMPORTANT METHODS BY CLASS

6.1 ArrayList

```
add(E e)
add(int index, E element)
get(int index)
set(int index, E element)
remove(int index) / remove(Object o)
contains(Object o)
size()
isEmpty()
clear()
indexOf(Object o)
lastIndexOf(Object o)
sort(Comparator<? super E> c)
```

6.2 LinkedList

```
add(E e)
addFirst(E e) / addLast(E e)
get(int index)
getFirst() / getLast()
remove(int index) / remove(Object o)
removeFirst() / removeLast()
offer(E e)
poll()
peek()
```

6.3 Vector

```
add(E e)
addElement(E obj)
get(int index)
set(int index, E element)
remove(int index)
removeElement(Object obj)
size()
capacity()
contains(Object o)
```

6.4 Stack

```
push(E item)
pop()
peek()
empty()
search(Object o)
```

LIFO → Last In First Out

6.5 HashSet

```
add(E e)
remove(Object o)
contains(Object o)
size()
isEmpty()
clear()
```

Does not allow duplicates and does not guarantee insertion order.

6.6 LinkedHashSet

```
add(E e)
remove(Object o)
contains(Object o)
size()
clear()
```

Does not allow duplicates and maintains insertion order.

6.7 TreeSet

```
add(E e)
remove(Object o)
contains(Object o)
first() / last()
higher(E e) / lower(E e)
ceiling(E e) / floor(E e)
pollFirst() / pollLast()
```

Stores unique elements in sorted order.

6.8 PriorityQueue

```
add(E e) / offer(E e)
peek()
poll()
remove(Object o)
contains(Object o)
size()
```

Priority is determined by the queue's ordering; iteration is not guaranteed to be sorted.

6.9 ArrayDeque

```
addFirst(E e) / addLast(E e)
offerFirst(E e) / offerLast(E e)
pollFirst() / pollLast()
peekFirst() / peekLast()
```

Double-ended queue.

6.10 HashMap

```
put(K key, V value)
get(Object key)
remove(Object key)
containsKey(Object key) / containsValue(Object value)
keySet()
values()
entrySet()
size()
isEmpty()
clear()
getOrDefault(Object key, V defaultValue)
```

Does not guarantee key iteration order.

6.11 LinkedHashMap

```
put(K key,V value)
get(Object key)
remove(Object key)
containsKey(Object key)
keySet()
values()
entrySet()
```

Maintains insertion order by default.

6.12 TreeMap

```
put(K key,V value)
get(Object key)
remove(Object key)
containsKey(Object key) / containsValue(Object value)
firstKey() / lastKey()
higherKey(K key) / lowerKey(K key)
ceilingKey(K key) / floorKey(K key)
entrySet()
```

Maintains sorted key order.

6.13 Hashtable

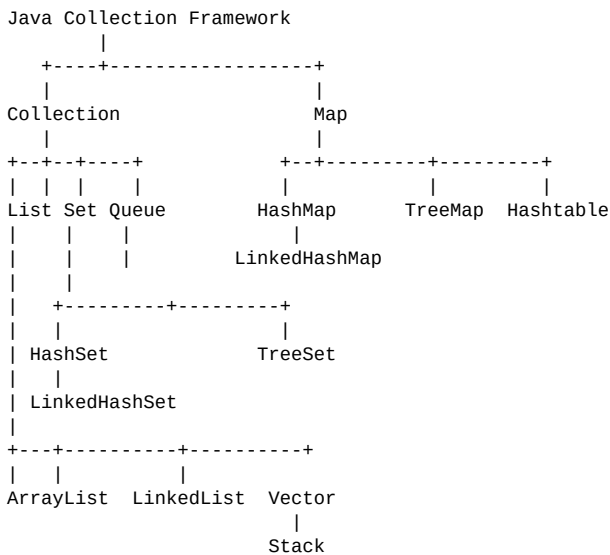
```
put(K key,V value)
get(Object key)
remove(Object key)
containsKey(Object key) / containsValue(Object value)
keys()
elements()
size()
isEmpty()
```

Legacy synchronized Map; does not permit null keys or null values.

7. ALGORITHM / LOGIC

1. Start the program.
2. Import java.util.*.
3. Create an ArrayList and demonstrate adding, accessing, updating, removing, searching and sorting.
4. Create a LinkedList and demonstrate list and queue operations.
5. Create a Vector and demonstrate its important methods.
6. Create a Stack and demonstrate push(), pop(), peek(), empty() and search().
7. Create HashSet, LinkedHashSet and TreeSet and demonstrate their differences.
8. Create a PriorityQueue and demonstrate priority-based insertion and removal.
9. Create an ArrayDeque and demonstrate operations at both ends.
10. Create HashMap, LinkedHashMap, TreeMap and Hashtable.
11. Demonstrate insertion, retrieval, deletion, searching and traversal of map elements.
12. Display the results.
13. Stop the program.

8. UML / COLLECTION CLASS STRUCTURE



Queue → PriorityQueue
Deque → ArrayDeque

9. COMPLETE JAVA PROGRAM

File Name: CollectionClassesDemo.java

```

import java.util.*;

public class CollectionClassesDemo {

    public static void main(String[] args) {

        // 1. ArrayList
        System.out.println("===== ARRAYLIST =====");

        ArrayList<String> list = new ArrayList<>();

        list.add("Java");
        list.add("Python");
        list.add("C");
        list.add(1, "C++");

        System.out.println("List: " + list);
        System.out.println("Get index 0: " + list.get(0));

        list.set(0, "Java Programming");
        System.out.println("After set: " + list);

        System.out.println("Contains Python: " + list.contains("Python"));
        System.out.println("Size: " + list.size());
        System.out.println("Index of C: " + list.indexOf("C"));

        list.add("Java Programming");
        System.out.println("Last index of Java Programming: "
            + list.lastIndexOf("Java Programming"));

        list.remove("C++");
    }
}
  
```

```

System.out.println("After remove: " + list);

list.sort(String.CASE_INSENSITIVE_ORDER);
System.out.println("After sorting: " + list);

// 2. LinkedList
System.out.println("\n===== LINKEDLIST =====");

LinkedList<String> linkedList = new LinkedList<>();

linkedList.add("Java");
linkedList.add("Python");
linkedList.addFirst("C");
linkedList.addLast("C++");

System.out.println("LinkedList: " + linkedList);
System.out.println("First: " + linkedList.getFirst());
System.out.println("Last: " + linkedList.getLast());
System.out.println("Get index 1: " + linkedList.get(1));

linkedList.offer("JavaScript");
System.out.println("After offer: " + linkedList);

System.out.println("Peek: " + linkedList.peek());
System.out.println("Poll: " + linkedList.poll());

linkedList.removeFirst();
linkedList.removeLast();

System.out.println("After removing first and last: "
    + linkedList);

// 3. Vector
System.out.println("\n===== VECTOR =====");

Vector<Integer> vector = new Vector<>();

vector.add(10);
vector.add(20);
vector.addElement(30);

System.out.println("Vector: " + vector);
System.out.println("Get index 1: " + vector.get(1));

vector.set(1, 25);
System.out.println("After set: " + vector);

vector.remove(0);
vector.removeElement(30);

System.out.println("After remove: " + vector);
System.out.println("Size: " + vector.size());
System.out.println("Capacity: " + vector.capacity());
System.out.println("Contains 25: " + vector.contains(25));

// 4. Stack
System.out.println("\n===== STACK =====");

Stack<Integer> stack = new Stack<>();

stack.push(10);
stack.push(20);
stack.push(30);

System.out.println("Stack: " + stack);
System.out.println("Peek: " + stack.peek());
System.out.println("Search 20: " + stack.search(20));
System.out.println("Pop: " + stack.pop());
System.out.println("Stack after pop: " + stack);
System.out.println("Empty: " + stack.empty());

// 5. HashSet
System.out.println("\n===== HASHSET =====");

HashSet<Integer> hashSet = new HashSet<>();

hashSet.add(30);
hashSet.add(10);
hashSet.add(20);
hashSet.add(10);

System.out.println("HashSet: " + hashSet);

```

```

System.out.println("Contains 20: " + hashSet.contains(20));
System.out.println("Size: " + hashSet.size());

hashSet.remove(10);
System.out.println("After remove 10: " + hashSet);
System.out.println("Is Empty: " + hashSet.isEmpty());

// 6. LinkedHashSet
System.out.println("\n===== LINKEDHASHSET =====");

LinkedHashSet<Integer> linkedHashSet = new LinkedHashSet<>();

linkedHashSet.add(30);
linkedHashSet.add(10);
linkedHashSet.add(20);

System.out.println("LinkedHashSet: " + linkedHashSet);
System.out.println("Contains 10: "
    + linkedHashSet.contains(10));
System.out.println("Size: " + linkedHashSet.size());

linkedHashSet.remove(20);
System.out.println("After remove 20: " + linkedHashSet);

// 7. TreeSet
System.out.println("\n===== TREESSET =====");

TreeSet<Integer> treeSet = new TreeSet<>();

treeSet.add(30);
treeSet.add(10);
treeSet.add(20);
treeSet.add(40);

System.out.println("TreeSet: " + treeSet);
System.out.println("First: " + treeSet.first());
System.out.println("Last: " + treeSet.last());
System.out.println("Higher than 20: " + treeSet.higher(20));
System.out.println("Lower than 20: " + treeSet.lower(20));
System.out.println("Ceiling of 20: " + treeSet.ceiling(20));
System.out.println("Floor of 20: " + treeSet.floor(20));

System.out.println("Poll First: " + treeSet.pollFirst());
System.out.println("Poll Last: " + treeSet.pollLast());
System.out.println("TreeSet after polling: " + treeSet);

// 8. PriorityQueue
System.out.println("\n===== PRIORITYQUEUE =====");

PriorityQueue<Integer> priorityQueue = new PriorityQueue<>();

priorityQueue.add(30);
priorityQueue.offer(10);
priorityQueue.add(20);

System.out.println("PriorityQueue: " + priorityQueue);
System.out.println("Peek: " + priorityQueue.peek());
System.out.println("Contains 20: " + priorityQueue.contains(20));
System.out.println("Size: " + priorityQueue.size());
System.out.println("Poll: " + priorityQueue.poll());
System.out.println("After poll: " + priorityQueue);

// 9. ArrayDeque
System.out.println("\n===== ARRAYDEQUE =====");

ArrayDeque<Integer> deque = new ArrayDeque<>();

deque.addFirst(20);
deque.addLast(30);
deque.offerFirst(10);
deque.offerLast(40);

System.out.println("Deque: " + deque);
System.out.println("Peek First: " + deque.peekFirst());
System.out.println("Peek Last: " + deque.peekLast());
System.out.println("Poll First: " + deque.pollFirst());
System.out.println("Poll Last: " + deque.pollLast());
System.out.println("Deque after polling: " + deque);

// 10. HashMap
System.out.println("\n===== HASHMAP =====");
HashMap<Integer, String> hashMap = new HashMap<>();

```

```

hashMap.put(101, "Raj");
hashMap.put(102, "Ravi");
hashMap.put(103, "Sita");

System.out.println("HashMap: " + hashMap);
System.out.println("Get key 101: " + hashMap.get(101));
System.out.println("Contains key 102: "
    + hashMap.containsKey(102));
System.out.println("Contains value Raj: "
    + hashMap.containsValue("Raj"));
System.out.println("Size: " + hashMap.size());
System.out.println("Keys: " + hashMap.keySet());
System.out.println("Values: " + hashMap.values());
System.out.println("Entries: " + hashMap.entrySet());

System.out.println("Get key 105: "
    + hashMap.getOrDefault(105, "Not Found"));

hashMap.remove(103);
System.out.println("After remove: " + hashMap);

// 11. LinkedHashMap
System.out.println("\n===== LINKEDHASHMAP =====");

LinkedHashMap<Integer, String> linkedHashMap =
    new LinkedHashMap<>();

linkedHashMap.put(101, "Raj");
linkedHashMap.put(102, "Ravi");
linkedHashMap.put(103, "Sita");

System.out.println("LinkedHashMap: " + linkedHashMap);
System.out.println("Get key 102: "
    + linkedHashMap.get(102));
System.out.println("Contains key 101: "
    + linkedHashMap.containsKey(101));
System.out.println("Keys: " + linkedHashMap.keySet());
System.out.println("Values: " + linkedHashMap.values());
System.out.println("Entries: " + linkedHashMap.entrySet());

linkedHashMap.remove(103);
System.out.println("After remove: " + linkedHashMap);

// 12. TreeMap
System.out.println("\n===== TREEMAP =====");

TreeMap<Integer, String> treeMap = new TreeMap<>();

treeMap.put(103, "Sita");
treeMap.put(101, "Raj");
treeMap.put(102, "Ravi");
treeMap.put(104, "Kiran");

System.out.println("TreeMap: " + treeMap);
System.out.println("Get key 101: " + treeMap.get(101));
System.out.println("First Key: " + treeMap.firstKey());
System.out.println("Last Key: " + treeMap.lastKey());
System.out.println("Higher Key than 102: "
    + treeMap.higherKey(102));
System.out.println("Lower Key than 102: "
    + treeMap.lowerKey(102));
System.out.println("Ceiling Key of 102: "
    + treeMap.ceilingKey(102));
System.out.println("Floor Key of 102: "
    + treeMap.floorKey(102));
System.out.println("Entries: " + treeMap.entrySet());

treeMap.remove(104);
System.out.println("After remove: " + treeMap);

// 13. Hashtable
System.out.println("\n===== HASHTABLE =====");

Hashtable<Integer, String> hashtable = new Hashtable<>();

hashtable.put(101, "Raj");
hashtable.put(102, "Ravi");
hashtable.put(103, "Sita");

System.out.println("Hashtable: " + hashtable);
System.out.println("Get key 101: " + hashtable.get(101));
System.out.println("Contains key 102: "

```

```

        + hashtable.containsKey(102));
System.out.println("Contains value Sita: "
        + hashtable.containsValue("Sita"));
System.out.println("Keys: " + hashtable.keySet());
System.out.println("Values: " + hashtable.values());
System.out.println("Size: " + hashtable.size());
System.out.println("Is Empty: " + hashtable.isEmpty());

hashtable.remove(103);
System.out.println("After remove: " + hashtable);
}
}

```

10. SAMPLE OUTPUT

```

===== ARRAYLIST =====
List: [Java, C++, Python, C]
Get index 0: Java
After set: [Java Programming, C++, Python, C]
Contains Python: true
Size: 4
Index of C: 3
Last index of Java Programming: 4
After remove: [Java Programming, Python, C, Java Programming]
After sorting: [C, Java Programming, Java Programming, Python]

===== LINKEDLIST =====
LinkedList: [C, Java, Python, C++]
First: C
Last: C++
Get index 1: Java
After offer: [C, Java, Python, C++, JavaScript]
Peek: C
Poll: C
After removing first and last: [Python, C++]

===== VECTOR =====
Vector: [10, 20, 30]
Get index 1: 20
After set: [10, 25, 30]
After remove: [25]
Size: 1
Capacity: 10
Contains 25: true

===== STACK =====
Stack: [10, 20, 30]
Peek: 30
Search 20: 2
Pop: 30
Stack after pop: [10, 20]
Empty: false

===== HASHSET =====
HashSet: [10, 20, 30]
Contains 20: true
Size: 3
After remove 10: [20, 30]
Is Empty: false

===== LINKEDHASHSET =====
LinkedHashSet: [30, 10, 20]
Contains 10: true
Size: 3
After remove 20: [30, 10]

===== TREESSET =====
TreeSet: [10, 20, 30, 40]
First: 10
Last: 40
Higher than 20: 30
Lower than 20: 10
Ceiling of 20: 20
Floor of 20: 20
Poll First: 10
Poll Last: 40
TreeSet after polling: [20, 30]

===== PRIORITYQUEUE =====
PriorityQueue: [10, 30, 20]
Peek: 10
Contains 20: true

```

```
Size: 3
Poll: 10
After poll: [20, 30]
```

```
===== ARRAYDEQUE =====
Deque: [10, 20, 30, 40]
Peek First: 10
Peek Last: 40
Poll First: 10
Poll Last: 40
Deque after polling: [20, 30]
```

```
===== HASHMAP =====
HashMap: {101=Raj, 102=Ravi, 103=Sita}
Get key 101: Raj
Contains key 102: true
Contains value Raj: true
Size: 3
Keys: [101, 102, 103]
Values: [Raj, Ravi, Sita]
Entries: [101=Raj, 102=Ravi, 103=Sita]
Get key 105: Not Found
After remove: {101=Raj, 102=Ravi}
```

```
===== LINKEDHASHMAP =====
LinkedHashMap: {101=Raj, 102=Ravi, 103=Sita}
Get key 102: Ravi
Contains key 101: true
Keys: [101, 102, 103]
Values: [Raj, Ravi, Sita]
Entries: [101=Raj, 102=Ravi, 103=Sita]
After remove: {101=Raj, 102=Ravi}
```

```
===== TREEMAP =====
TreeMap: {101=Raj, 102=Ravi, 103=Sita, 104=Kiran}
Get key 101: Raj
First Key: 101
Last Key: 104
Higher Key than 102: 103
Lower Key than 102: 101
Ceiling Key of 102: 102
Floor Key of 102: 102
Entries: [101=Raj, 102=Ravi, 103=Sita, 104=Kiran]
After remove: {101=Raj, 102=Ravi, 103=Sita}
```

```
===== HASHTABLE =====
Hashtable: {103=Sita, 102=Ravi, 101=Raj}
Get key 101: Raj
Contains key 102: true
Contains value Sita: true
Keys: [103, 102, 101]
Values: [Sita, Ravi, Raj]
Size: 3
Is Empty: false
After remove: {102=Ravi, 101=Raj}
```

Note: HashSet, HashMap and Hashtable do not guarantee iteration order, so their displayed order may vary between executions.

11. COMPILATION AND EXECUTION

```
javac CollectionClassesDemo.java
java CollectionClassesDemo
```

12. REAL-TIME APPLICATIONS

ArrayList	Student lists, product lists
LinkedList	Browser history, playlists
Vector	Legacy synchronized applications
Stack	Undo operations, expression evaluation
HashSet	Removing duplicate data
LinkedHashSet	Unique data with insertion order
TreeSet	Sorted unique data

PriorityQueue	Task scheduling, priority processing
ArrayDeque	Queue and stack operations
HashMap	Student ID → Student Name
LinkedHashMap	Ordered records
TreeMap	Sorted key-value records
Hashtable	Legacy thread-safe key-value systems

13. RESULT

Thus, the important methods of various Java Collection Framework classes such as ArrayList, LinkedList, Vector, Stack, HashSet, LinkedHashSet, TreeSet, PriorityQueue, ArrayDeque, HashMap, LinkedHashMap, TreeMap, and Hashtable were studied and successfully implemented using Java programs.